Seasonal Farm Market: An Unbreakable RL Environment

Hackathon: Build an RL Environment That Can't Be Broken | Team: prple (team_148)

- Core Mission: Build a world an AI agent can act in, get scored by, and fail to cheat.
- Domain Track: Games & World Simulations (Running a realistic agricultural economy under dynamic market constraints).
- Technology Foundation: Pure Python Standard Library (0 third-party packages in sandbox, <30s Docker build).
- Dual-Container Physical Isolation: Sealed HTTP sandbox (internal: true, no internet, read-only FS) + replay verification harness.
- Standardized 20-Day Horizon: Uniform 20-day horizon across all 31 tasks (4 plots, 3 crops, perishable spoilage, seasonal cycles).
- Verified Reliability: 3,100/3,100 automated gate checks pass across 31 tasks and 50 seeds with zero flakiness.

The Challenge: Why RL Environments Break

Agents are relentless optimizers that exploit weak reward structures, float drift, and leaky state.

- Common Weakness 1: Tiny state spaces (Tic-tac-toe/grid-worlds) that models memorize or solve in trivial brute-force steps.
- Common Weakness 2: LLM-as-judge or fuzzy regex checkers that award undeserved points for hallucinated or sycophantic text.
- Common Weakness 3: Leaky sandboxes exposing bash/read_file where models inspect hidden unit tests or edit score files directly.
- Common Weakness 4: Floating-point rounding loopholes, infinite debt exploitation, or hoarding unsellable inventory for phantom gains.
- Common Weakness 5: Inconsistent task horizons and uncompressed context causing agent disorientation or token window overflows.
- Our Philosophy: Defense-in-depth architectural isolation combined with deterministic microeconomics and integer cent arithmetic.

What We Did Different: Engineering Against Exploits

A direct architectural comparison: Conventional RL Hackathon setups vs. Team Prple's defended design.

Architectural Pillar	Conventional RL Setup (Vulnerable)	Team Prple Defended Implementation
Sandbox Security	Single container, bash/exec access, filesystem read/write, open internet	Dual-container: sealed sandbox, read-only FS, zero shell tools, 11 typed RPCs
Scoring Integrity	In-sandbox evaluation script or LLM-as-judge vulnerable to prompt injection	Out-of-band replay harness: bit-for-bit re-execution against fresh engine
Reward Arithmetic	Floating-point floats with rounding drift; phantom score from hoarded stock	Strict integer cents ($45 = 4500c), min(floor, market) anti-hoarding valuation
Horizon Consistency	Variable / disjoint horizons (10-60 days) causing agent disorientation	Standardized 20-day horizon across all 31 tasks; context kept <12K tokens
Microeconomic Depth	Trivial grid-worlds or static linear games memorized in brute-force steps	5 structural microeconomic traps (Cobweb, Hotelling, Cournot, Dutch disease)
Live Observability	Headless batch console logs with zero interactive testing or visual insight	1-command Docker Web Studio (:8500) with animated SVG crops, charts & ledger

Architectural Takeaway: Rather than hoping agents behave, we isolate execution physically, eliminate math exploits deterministically, and provide a full live visual studio for transparent human and automated evaluation.

Dual-Container Zero-Trust Architecture

Physical runtime isolation between untrusted agent execution and deterministic scoring verifier.

- Sealed Sandbox Container: Docker network internal: true (no internet access), read-only root filesystem, dropped capabilities (cap_drop: ALL, no-new-privileges).
- Zero Attack Surface: Sandbox exposes strictly 11 domain HTTP tools (plant, harvest, sell, loan, advance_days, etc.); zero bash, python exec, or file tools.
- Isolated Orchestration Harness: Connects to gpt-6-luna via litellm gateway, enforces 40-step budget, 2KB tool caps, and history compression.
- Cryptographic Out-of-Band Scoring: Sandbox never computes final score; harness replays verified action log against fresh engine.
- Ultra-Fast Clean Build: Pinned Python 3.11-slim standard library; clean Docker build completes in <30 seconds from zero cache.

Microeconomic Depth & Standardized 20-Day Horizon

31 varied tasks across 4 difficulty tiers + procedural seed variation generating hundreds of scenarios.

- Standardized 20-Day Horizon: All 31 tasks unified to a 20-day horizon, eliminating time-horizon dissonance across benchmarks.
- Easy Tier (farm_basic, farm_two_crops, farm_sprint): Core planting/harvesting, grow-time vs. yield tradeoffs, cash velocity.
- Medium Tier (farm_seasons, farm_events, farm_spoilage): Seasonal sinusoidal prices, shelf-life spoilage, storage caps, weather shocks.
- Hard Tier (farm_credit, farm_crash, farm_drought): Compounding interest, credit limits, order slippage, noisy forecasts.
- Deep Microeconomic Benchmarks: Cobweb cycles (Ezekiel 1938), Dutch disease resource curse, Hotelling scarcity arbitrage, Cournot slippage.
- Procedural Variation by Seed: Seeds vary base prices (+/-15%), seed costs (+/-10%), yields (+/-1), shock timing, and seasonal phases.
- Dynamic Oracle Scaling: Oracle solves each procedural seed dynamically; 3,100/3,100 gate checks pass with oracle >= 0.95 and no-op <= 0.05.

Deterministic Programmatic Reward Design

Integer cents, 0.95x oracle scaling, anti-hoarding valuation, and strict debt penalty.

- Net Worth Formulation: Cash + sum(Inventory * min(floor_price, current_market_price)) - Debt.
- Anti-Hoarding Valuation: Unsold crops evaluated at min(floor, market). If market crashes below floor, zero phantom value is created.
- Continuous Non-Saturating Gain: profit scales monotonically with independent oracle benchmark (> 0.95 and < 1.0) without false ceilings.
- Saturation Eliminated: Non-clamped scoring cleanly separates naive attempts (0.20-0.40) from skilled strategic runs (0.85-0.98).
- Insolvent Debt Penalty: total_score = profit * (0.85 + 0.15 * debt_ok). Unpaid debt imposes a strict 15% penalty haircut.
- Daily Land Tax Drain: Land taxes apply daily ($15-$25/plot/day). Inaction (no-op) erodes capital and guarantees score <= 0.05.
- Zero Rounding Arbitrage: All economic state is tracked strictly in integer cents (e.g., $45.00 = 4500 cents).

Red Team Adversarial Attack Matrix

Empirical verification across 7 adversarial attacker vectors (scripts/red_team.py) - 100% Contained & Defended.

Attack Vector	What It Tests	What Happened (Observed)	Engine Defense Mechanism
Hoarder Agent	Unsold inventory gives unearned score	Scored 0.0000; phantom value wiped	Valued at min(floor, market) (50% base). Taxes drain cash. Score < 0.60.
Counterfeit Selling	Selling > owned or negative quantity	Rejected with error; cash unchanged	Strict bounds (qty in [1, owned]); rejects negative/zero sales before mutation.
Deadbeat Borrower	Take max loan and never repay	Scored 0.0000; total score crushed	Debt deducted from net worth; 0.85 debt_ok multiplier; compounding interest.
TimeWarp / Null Args	advance_days(9999), null, bad types	Clamped to 7 days; 0 server crashes	Strict type coercion & [1, 7] bounds validation; invalid actions safely rejected.
Plot Squatter / Fake Crop	Plant on occupied plot or fake crop	Rejected with error; plot preserved	Plot occupancy guard & crop catalog lookup; state unmodified; episode proceeds.
Tool Flooder	Spam 100 tool calls in single reply	Calls 11-100 dropped; security alert	Hard cap MAX_TOOL_CALLS_PER_REPLY = 10; memory & execution time protected.
Garbage Speaker	Truncated JSON, raw conversational text	Parsed valid candidates; 0 crashes	Dual-mode parser with regex fallback & normalizer; resilient token extraction.

Red Team Outcome: All 7 adversarial exploits defeated. Suspicious action counter logs [SECURITY WARNING] alerts.

10,000 Invariant Fuzzing & Cryptographic Replay

Extensive random action stress-testing and bit-for-bit replay verification tool (scripts/verify_replay.py).

- 10,000-Step Adversarial Invariant Fuzzing (scripts/fuzz_engine.py):
  * Tested 10,000 random and adversarial action sequences across 104+ full episodes.
  * Invariant 1: Cash never silently goes negative without debt record.
  * Invariant 2: Inventory quantities are strictly non-negative (>= 0) at all times.
  * Invariant 3: Determinism: Same seed + same actions yields 100% identical SHA-256 state hash.
  * Fuzzing Outcome: 0 unhandled crashes, 0 invariant violations across all 10,000 steps.
- Cryptographic Replay Verification Tool (scripts/verify_replay.py):
  * Independent CLI tool reads episode.jsonl, replays logged actions through a fresh engine, and compares hashes turn-by-turn.
  * Verified 100% bit-for-bit match on sample-trajectories/success (8/8 steps) and failure (9/9 steps).

Model Separation Benchmark & Empirical Economics

Monotonic capability spread across models and exact measured numbers from the real simulation engine.

Model Capability Separation Benchmark (scripts/benchmark_separation.py)

Scenario	Weak	Mid	Strong	Spread
farm_basic	0.0000	0.3623	1.0000	+0.6377
farm_seasons	0.0000	0.1352	1.0000	+0.8648
farm_events	0.0000	0.0646	1.0000	+0.9354
farm_credit	0.0000	0.0219	1.0000	+0.9781
econ_cobweb	0.0000	0.0000	1.0000	+1.0000
econ_hotelling	0.0000	0.0437	1.0000	+0.9563

Empirical Economic Trap Benchmarks (scripts/verify_economics.py)

Economic Benchmark	Naive Score	Smart Score	Spread
Cournot Elasticity	$73.27 (0.24)	$171.04 (1.00)	+0.7639
Cobweb Cycle	$44.69 (0.00)	$282.72 (1.00)	+1.0000
Liquidity Mismatch	$-15.54 (0.00)	$143.76 (1.00)	+1.0000
Hotelling Arbitrage	$39.15 (0.00)	$159.43 (1.00)	+1.0000
Dutch Disease	$23.40 (0.00)	$194.68 (1.00)	+1.0000
Pigouvian Commons	$29.40 (0.00)	$157.29 (1.00)	+1.0000

Conclusion: Clear monotonic separation across models. No ceiling/floor collapse. Empirical numbers measured directly in real engine.

Model-Agnostic Resilient Harness & Live Limits

Dual-mode parsing, 40-step budget, tool output caps, and observation compression (<64K tokens).

- Any OpenAI-Compatible Model: Reads MODEL, API_KEY, BASE_URL. Tested with gpt-6-luna and 4 diverse mock agents.
- Dual-Mode Parsing: Parses native OpenAI tool_calls or extracts fenced JSON from raw markdown text with automatic normalization.
- Live Limits Fit: Standardized 20-day horizon is finishable in under 20 steps via advance_days(1-7) and multi-tool batching.
- Tool Output Capping: All tool execution responses are hard-capped at 2,000 characters, preventing verbose memory dumps.
- Turn History Compression: Preserves the last 3 turns with complete detail; compresses older turns into 1-line state digests.
- Context Length Guarantee: 20-day episodes consume <12K tokens, staying well beneath the 64K token limit with zero overflow risk.

Interactive Web Studio & Live Dashboard (GUI)

Standalone full-stack web simulator and evaluation console running on http://localhost:8500.

- 1-Command Launch: docker compose up --build boots the interactive visual simulator instantly on port 8500.
- Real-Time Dynamic Graphics: Animated SVG plots reflecting crop growth stages (Seedling, Growing, Mature, Withered) and weather.
- Interactive Farming Sandbox: Evaluators can play through any scenario manually, step-by-step, to test economic dynamics and edge cases.
- Live Financial Ledgers & Charts: Real-time Cash, Debt, Net Worth, Market Pricing trends, and crop inventory trackers.
- High-Contrast Polished UI: Obsidian/dark theme with pristine high-contrast white metrics, step-by-step action history, and reset controls.
- Dual Operational Profiles: Default launches interactive Studio on port 8500; docker compose --profile eval up runs headless automated evaluation.

Conclusion & Submission Checklist

Production-ready for live VM execution on main branch | Team prple (team_148).

- One Command Execution: docker compose up --build (Dashboard :8500) or docker compose --profile eval up (Evaluation Harness).
- Ultra-Fast Clean Build: Pinned Python stdlib image builds in <30 seconds from clean cache with zero external packages in sandbox.
- Sealed Security: Sandbox has no internet, no shell tools, and cannot tamper with verifier replay scoring.
- Full Deliverables Included:
  * baseline-results/gate.json (3,100 / 3,100 gate checks passed with 100% reliability)
  * test-run-logs/ (3 full runs: basic, demo, and chaos stress)
  * sample-trajectories/ (pass/fail trajectories with 100% bit-for-bit replay verification)
  * design-doc.pdf (strictly 2 pages) and deck.pdf (strictly 12 slides)
  * scripts/ (verify_replay.py, red_team.py, fuzz_engine.py, verify_economics.py, benchmark_separation.py, dashboard.py)
- Ready for judges evaluation. Thank you!